

KLITE RPmod — User Guide

KLITE RPmod adds roleplay tooling to **Esobold Esolite** (a KoboldAI Lite fork): character panels, a guided "Easy mode" for beginners, and the **Worlds** system — a living, graph-based world your story can move through. This guide focuses on using it, with an emphasis on the **in-chat Worlds commands**.

1. Installing

The mod ships as a single file, `KLITE-RPmod.js`.

1. In Esolite, open the **mod manager** (settings → user mod).
2. Load / paste `KLITE-RPmod.js` and enable it. Reload when prompted.
3. After it loads you'll see the existing RPmod panels, plus a  **Worlds** button in the navbar and a small **Worlds panel** in the bottom-right corner.

That's it — one file contains everything (main panels, guided onboarding, the Worlds engine, and the Worlds editor).

Try it in one click. Open the Worlds panel and press  **Load example world** — it loads a ready-to-play world (a village, a tavern, a guarded frontier road, quests with givers, an event chain, and a bandit encounter), enables it, and drops you in. Just start chatting. Everything below explains how to build your own.

2. What you get

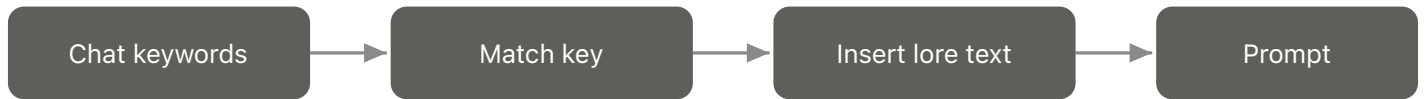
- **Right-side panels** — CHARS, ROLES, TOOLS, CONTEXT, IMAGES: manage characters, personas and groups, context tools, and image generation.
 - **Guided RP** — a beginner-friendly step-by-step setup overlay (toggleable with an Advanced mode for the full interface).
 - **Worlds** — the focus of this guide, below.
-

3. What is Worlds?

A classic lorebook dumps in any entry whose keyword appears in the chat — with no sense of *where* or *when* you are. Worlds instead models a real world (locations, NPCs,

factions, objects, events, a clock) and feeds the AI **only the slice relevant to your current location and time**.

Classic lorebook — keyword → text



Every matched entry is dumped in, no sense of place or time.

Worlds — state → relevant slice



Only the current location's characters, objects, events and relevant lore — not the whole world.

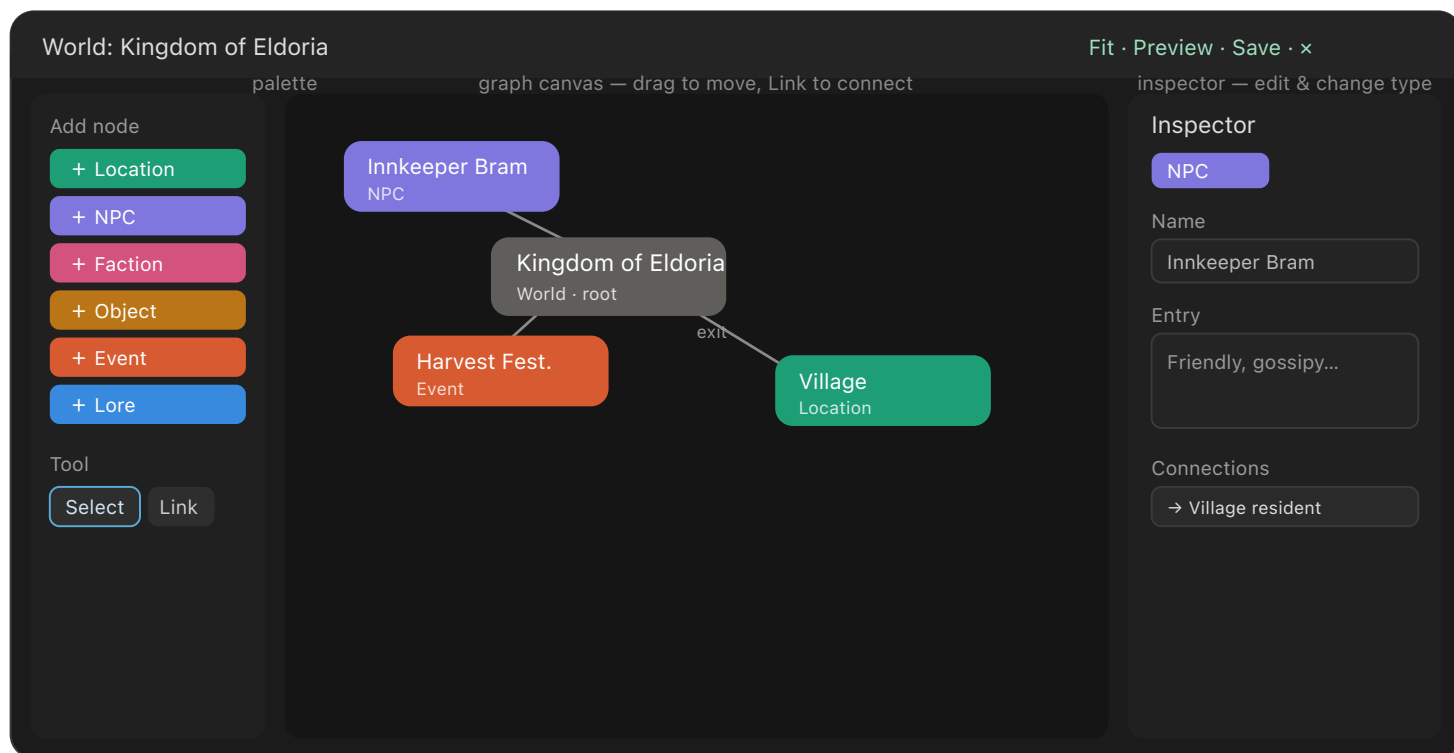
Result: less prompt bloat, better consistency, real location & time awareness.

The result: less prompt bloat, better consistency, and genuine location & time awareness. Persistent NPCs move on schedules, events fire when their conditions are met, and the world remembers state (flags, inventory, quest progress) as you play.

Under the hood the mod hands Esolite a compact, per-turn summary through its normal WorldInfo pipeline — you don't have to manage any of that; it's automatic.

4. Building a world (the editor)

Open the editor with the  **navbar button** or **Open editor** in the Worlds panel.
(No world yet? It'll offer to create one.)



- **Palette (left)** — click a type to drop a colour-coded node: Location · NPC · Faction · Object · Event · Lore.
- **Canvas (middle)** — drag nodes to arrange them. Wheel to zoom, drag empty space to pan, **Fit** to reframe.
- **Tools** — **Select** (move nodes) and **Link** (click one node, then another, to connect them).
- **Inspector (right)** — edit the selected node's name, entry, and type-specific fields; review/remove its connections; change its **type**; or delete it.
- **Save** writes the world to your library; **Preview** shows exactly *what the AI will* see for your current location.

Connections carry meaning

You don't label arrows — the editor infers the relationship from the two node types and writes it into the data:

Drag between	Becomes
Location ↔ Location	an exit (both directions)
NPC → Location	NPC lives / is stationed there
NPC → Faction	NPC belongs to that faction
Object → Location	object is located there
Object → NPC	object is owned by that NPC

Event → Location

event **can occur** there

So the graph you draw *is* what the AI traverses. Invalid pairs (e.g. NPC → NPC) are rejected.

Tip: after **importing** a lorebook (section 8) everything arrives as grey **Lore** nodes. Select one and use the inspector's **type** dropdown to promote it to a Location or NPC, then wire up exits.

5. Playing with a world

From the **Worlds panel** (bottom-right):

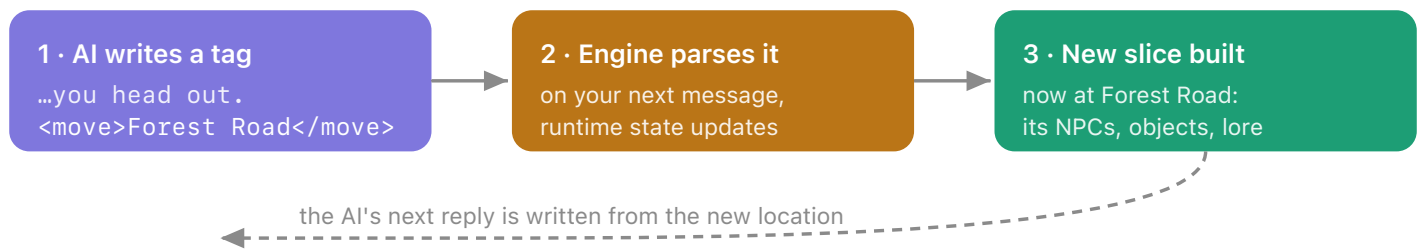
- Pick a world from the dropdown (or **+ New**).
- **Enable for this story** — turns the world on for the current chat. (Off = Esolite behaves normally; nothing is injected.)
- **Current location** — set where the player is; the slice follows it.
- **Clock** — shows day / time / season, with **⏭** to advance time.

Now just chat. Each turn, the AI receives your current location, the NPCs and objects there, active events, and relevant lore — and nothing from the far side of the map.

6. In-chat commands (Worlds tags)

The world changes as you play through small **tags**. The AI can emit them in its replies (instruct it to, via your World Rules), or **you can type them yourself**. They're parsed on your next turn, update the world state, and are reflected in the following slice.

How an in-chat command changes the world



Categories

- movement <move> <npcmove>
- characters <mood>
- state <flag> <quest> items
- time <time> <weather> <advance>
- inventory <give> <take>

Tip: you can type these yourself too — e.g. send <move>Tavern</move> to relocate.

Command reference

Command	Example	Effect
<move>...</move>	<move>Forest Road</move>	Move the player to a location
<npcmove>NPC=Loc</npcmove>	<npcmove>Captain Rowan=Village</npcmove>	Move an NPC to a location
<mood>NPC=Mood</mood>	<mood>Bram=cheerful</mood>	Set an NPC's mood
<flag>key=value</flag>	<flag>metRowan=true</flag>	Set a story flag (<flag>key</flag> = true)
<unflag>key</unflag>	<unflag>metRowan</unflag>	Clear a flag
<give>Item</give>	<give>Torch x2</give>	Add to inventory (x2 optional)
<take>Item</take>	<take>Torch</take>	Remove from inventory
<quest>id=state</quest>	<quest>find_sword=active</quest>	Set a quest's state (done hides it)
<time>slot</time>	<time>evening</time>	Set time of day
<weather>...</weather>	<weather>rain</weather>	Set the weather
<advance>	<advance>	Advance the clock one step

<code><action>text</action></code>	<code><action>pick the lock</action></code>	Fire an action signal (triggers <code>onAction</code> events)
<code><roll>expr</roll></code>	<code><roll>1d20+3</roll></code>	Roll dice (logged in combat)
<code><attack>A-&gt;B</attack></code>	<code><attack>You-&gt;Goblin</attack></code>	Resolve an attack (to-hit vs AC, damage, HP)
<code><hp>Name=±N</hp></code>	<code><hp>Goblin=-4</hp></code>	Adjust a combatant's HP
<code><check>Name=abi DC</check></code>	<code><check>You=dex 12</check></code>	Ability check vs a DC

Time slots: `morning → noon → afternoon → evening → night` . Advancing past night rolls to the next day, and the **season** follows the month automatically.

Notes

- Tags are applied at the **start of your next message**, so a move the AI narrates this turn takes effect from the next turn onward — which reads naturally in the story.
- Tags currently remain visible in the chat text (they're read, not hidden). To have the AI use them, add a line to your **World Rules** such as: *"When the scene changes location, emit `<move>Name</move>` ."*
- Want time to pass automatically every turn? Enable it once in the console:
`KLITE_RPMod_Worlds.config.advanceClockPerTurn = true` .

6b. Tabletop-RPG systems

The Worlds panel has four tabs — **Play**, **Quests**, **Combat**, **Editor** — and a **Creator** ⇌ **Player** lens in its header.

State slots (base / working). The Play tab keeps two saved states: **base** (your start point) and **working** (the live game). Use **Reset** to snap back to base, **Commit** to make the current state the new base, and **Swap** to switch which is active. Both travel with your save and export.

Persons = characters. In the editor, an NPC can be **linked to a character** from your CHARS library (its TavernCard text is reused) and given an optional **d20 stat block** (abilities, AC, HP, attacks). Stats feed combat and appear in the AI's context near that NPC.

Quests (WoW-style). Add **Quest** nodes with a **giver** (!) and **turn-in** person (?) — those markers show on the map and persons. The **Quests tab** is your log: accept, complete, turn in, and track quests; hidden quests read ??? to the player until discovered. A per-world switch controls whether the **AI** (as GM) sees hidden content or not.

Triggers & chains. Events have **triggers** (on enter / time / flag / quest-state / action / another event) and **effects** (set flags, give items, offer quests, move NPCs, fire another event...). Effects chain, so you can build sequences like *enter the tavern at night → a courier arrives → he offers a delivery quest*. Wire it all visually in the editor.

Factions & HQs. Link a **Faction** → **Location** to give it a visitable **headquarters**.

Combat (d20 / SRD 5.1). In the **Combat tab**, pick combatants (quick-add SRD monsters like goblin/wolf/skeleton), **Start encounter** to roll initiative, then attack, roll dice, adjust HP, and step through turns. The mod is authoritative for the maths; the current combat state (round, turn order, HP) is injected so the AI narrates the outcomes. You or the AI can also drive combat with the <attack>, <roll>, <hp> and <check> tags above.

7. What the AI actually sees

Click **Preview** in the editor (or run `KLITE_RPMod_Worlds.preview()` in the console) to see the assembled slice, e.g.:

```
[World Rules]
Medieval fantasy tone.

[Current Time]
Day 1, spring (morning). Weather: clear.

[Current Location: Village]
A small farming village.
Exits: Forest Road

[Nearby NPCs]
– Innkeeper Bram | friendly, gossipy

[Relevant Lore]
The King rules from the distant capital.
```

Sections are prioritised (rules, time, player state, location, NPCs, objects, events, lore) and trimmed to fit your context budget, so the important things survive.

8. Import & export

In the **Worlds** panel:

- **↓ Import** — load a classic WorldInfo / lorebook / TavernCard `character_book` JSON. Entries become **Lore** nodes you can promote to Locations/NPCs in the editor. You'll be asked whether to merge into the active world or create a new one.
- **↑ Export** — save the world as portable **World JSON**, or down-convert to a flat **WorldInfo** file compatible with vanilla Lite.

Worlds are stored in your browser (IndexedDB); the per-story runtime state (where you are, flags, inventory, clock) is saved inside the story savefile automatically. Your saved stories never contain the temporary injected entries.

9. Power-user console commands

Everything is scriptable via `KLITE_RPMod_Worlds` (alias `W` below):

```
const W = KLITE_RPMod_Worlds;
W.newWorld('Eldoria');           // create + activate a world
W.openEditor();                  // open the graph editor
W.enable(); W.disable();         // per-story on/off
W.moveTo('Village');             // move the player
W.advanceClock(2);                // pass two time steps
W.setClock({ season: 'autumn' }); // set clock fields
W.giveItem('Torch', 2);          // inventory
W.setFlag('metRowan', true);     // flags
W.setQuest('find_sword', 'active');
W.applyTags('<move>Tavern</move>'); // apply tag text directly
W.preview();                     // the current slice, as a string
W.exportWorld();                 // portable JSON
W.exportWorldAsWI();             // flat WorldInfo array
```

10. Tips & troubleshooting

- **Nothing is injected?** Make sure a world is selected **and** "Enable for this story" is on, and that you've set a current location.
- **AI ignores locations?** Add explicit guidance in **World Rules**, and give locations a short description so they read clearly.
- **A location has no detail** — that's fine; the `[Current Location: ...]` header always appears so the AI still knows where it is.

- **Events not firing** — events trigger on authored conditions (time/season/flags). Basic events can be placed in the editor; detailed conditions/effects are set via imported data or the console for now.
- **Start over for a chat** — just toggle the world off, or switch to a different world in the panel.